Game 2.0

Manuel Gherardi

RPG 2D Action Adventure

Progetto del corso di OOP riscritto con le regole del corso di **Ingegneria del Software**.
Repository Game(OOP): github.com/GHManu/Game,
Repository Game2.0(Ing Del Sw): github.com/GHManu/Game2.0,

Nota: I codici trascritti e i vari diagrammi UML **non rappresentano l'intero progetto ma solo la parte significativa di esso**, nelle classi rappresentate nel Class Diagram non ho messo tutte le variabili/metodi, stessa cosa per il Sequence, poichè il progetto anche se semplice è abbastanza complesso; quindi l'ho semplificato rappresentando il fulcro e le parti più importanti del software.

Dichiaro che questo elaborato è frutto del mio personale lavoro, svolto sostanzialmente in maniera individuale e autonoma.

Documento di Analisi dei Requisiti e Design

Indice

1 Analisi dei Requisiti **2**
- 1.1 Scopo . . . 2
- 1.2 Requisiti Funzionali . . . 2
  - 1.2.1 RF1 – Movimento del giocatore . . . 2
  - 1.2.2 RF2 – Attacco del giocatore . . . 2
  - 1.2.3 RF3 – Comportamento del nemico . . . 2
  - 1.2.4 RF4 – Gestione della mappa . . . 2
  - 1.2.5 RF5 – Sistema di salute e condizioni di vittoria/sconfitta . . . 3
  - 1.2.6 RF6 – Interfaccia utente . . . 3
- 1.3 Requisiti Non Funzionali . . . 3
  - 1.3.1 RNF1 – Usabilità . . . 3
  - 1.3.2 RNF2 – Affidabilità . . . 3
  - 1.3.3 RNF3 – Manutenibilità . . . 3
  - 1.3.4 RNF4 – Portabilità . . . 4
  - 1.3.5 RNF5 – Sicurezza . . . 4
  - 1.3.6 RNF6 – Qualità grafica . . . 4
  - 1.3.7 RNF7 – Scalabilità . . . 4

2 Design Pattern e Modifiche Apportate **4**
- 2.1 Convenzioni di Nomenclatura . . . 4
- 2.2 Strategy Pattern – Personaggi . . . 4
- 2.3 Builder – Player ed Enemy . . . 5
- 2.4 Factory Method – Armi e Personaggi . . . 6
- 2.5 Iterator – Proiettili . . . 7
- 2.6 MVC – Menù e UI . . . 7
- 2.7 State Pattern – Game Loop e UI . . . 8
- 2.8 Singleton – HUD . . . 9
- 2.9 Command Pattern – Tasti UI . . . 9

3 UML – Use Case Diagram **11**

4 UML – State Diagram **11**

5 UML – Activity Diagram **12**

6 UML – Class Diagram **13**

7 UML – Sequence Diagram **14**

Analisi dei Requisiti

Scopo

L'obiettivo del gioco è sconfiggere il nemico muovendosi tramite `WASD` o frecce direzionali, con `Shift` per lo sprint temporaneo con ricarica in una mappa piccola e chiusa, attaccando tramite il tasto sinistro del mouse. Il nemico si muove in due direzioni e attacca con un proiettile alla volta: quando uno esplode, ne parte un altro.

Requisiti Funzionali

RF1 – Movimento del giocatore

- **RF1.1** – Il sistema deve permettere al giocatore di muovere il personaggio nelle quattro direzioni tramite i tasti `WASD` o le frecce direzionali.
- **RF1.2** – Il sistema deve permettere al giocatore di attivare uno sprint temporaneo tramite il tasto `Shift`.
- **RF1.3** – Il sistema deve gestire un tempo di ricarica (*cooldown*) dello sprint, impedendone l'uso finché non è ricaricato.
- **RF1.4** – Il sistema deve impedire al giocatore di uscire dai confini della mappa.

RF2 – Attacco del giocatore

- **RF2.1** – Il sistema deve permettere al giocatore di attaccare tramite il tasto sinistro del mouse.
- **RF2.2** – Il sistema deve rilevare le collisioni tra l'attacco del giocatore e il nemico.
- **RF2.3** – Il sistema deve applicare danno al nemico quando viene colpito.

RF3 – Comportamento del nemico

- **RF3.1** – Il sistema deve generare almeno un nemico all'interno della mappa.
- **RF3.2** – Il nemico deve muoversi automaticamente in due direzioni predefinite (es. avanti/indietro, sinistra/destra).
- **RF3.3** – Il nemico deve attaccare il giocatore.
- **RF3.4** – Il sistema deve rilevare le collisioni tra il proiettile nemico e il giocatore.
- **RF3.5** – Il sistema deve applicare danno al giocatore quando viene colpito.

RF4 – Gestione della mappa

- **RF4.1** – Il sistema deve caricare una mappa chiusa e di dimensioni limitate.
- **RF4.2** – Il sistema deve impedire a giocatore e nemico di attraversare i muri o gli ostacoli.
- **RF4.3** – Il sistema deve gestire la posizione iniziale del giocatore e del nemico.

RF5 – Sistema di salute e condizioni di vittoria/sconfitta

- **RF5.1** – Il sistema deve gestire i punti vita del giocatore.
- **RF5.2** – Il sistema deve gestire i punti vita del nemico.
- **RF5.3** – Il sistema deve dichiarare la vittoria quando il nemico viene sconfitto.
- **RF5.4** – Il sistema deve dichiarare la sconfitta quando i punti vita del giocatore raggiungono zero.
- **RF5.5** – Il sistema deve mostrare un messaggio o una schermata di fine partita.

RF6 – Interfaccia utente

- **RF6.1** – Il sistema deve mostrare la barra della vita del giocatore.
- **RF6.2** – Il sistema deve mostrare la barra della vita del nemico (se previsto).
- **RF6.3** – Il sistema deve mostrare lo stato dello sprint (attivo, in ricarica, disponibile).
- **RF6.4** – Il sistema deve permettere di riavviare la partita dopo la fine.

Requisiti Non Funzionali

RNF1 – Usabilità

- **RNF1.1** – I comandi devono essere intuitivi e facilmente accessibili (`WASD`/frecce, mouse, `Shift`).
- **RNF1.2** – L'interfaccia deve essere chiara e mostrare informazioni essenziali (vita, sprint, vita del nemico).
- **RNF1.3** – Il gioco deve essere comprensibile senza necessità di un tutorial complesso.

RNF2 – Affidabilità

- **RNF2.1** – Il sistema non deve andare in crash durante una sessione di gioco standard.
- **RNF2.2** – Le collisioni devono essere rilevate in modo coerente e ripetibile.
- **RNF2.3** – Il comportamento del nemico deve essere deterministico o comunque privo di errori logici (es. non deve sparare due proiettili contemporaneamente).

RNF3 – Manutenibilità

- **RNF3.1** – Il codice deve essere strutturato in moduli separati .
- **RNF3.2** – Il sistema deve permettere l'aggiunta di nuovi nemici o nuove abilità senza modificare pesantemente il codice esistente.
- **RNF3.3** – Il progetto deve includere documentazione minima per facilitare modifiche future.

RNF4 – Portabilità

- **RNF4.1** – Il gioco deve essere eseguibile su tutti i sistemi operativi .
- **RNF4.2** – Il sistema non deve dipendere da librerie non portabili o non supportate.

RNF5 – Sicurezza

- **RNF5.1** – Il sistema deve impedire input non validi o fuori dai limiti (es. coordinate negative, velocità infinita).
- **RNF5.2** – Il sistema deve evitare exploit che permettano al giocatore di uscire dalla mappa o attaccare più velocemente del previsto.

RNF6 – Qualità grafica

- **RNF6.1** – Le animazioni devono essere fluide e coerenti con lo stile del gioco.
- **RNF6.2** – Gli asset grafici devono essere leggibili e non confondere il giocatore.

RNF7 – Scalabilità

- **RNF7.1** – Il sistema deve poter gestire l'aggiunta di ulteriori elementi (nuovi proiettili, nuovi nemici, ostacoli) senza degradare significativamente le prestazioni.
- **RNF7.2** – La mappa deve poter essere sostituita o ampliata senza riscrivere il motore di gioco.

Design Pattern e Modifiche Apportate

Convenzioni di Nomenclatura

- Le **classi astratte** sono prefissate con `A`.
- Le **interfacce** sono prefissate con `I`.
- Le **enumerazioni** sono prefissate con `E`.
- I **metodi** seguono il *camelCase*.
- Le **variabili** sono tutte minuscole con separatore `_`.
- Le **costanti** sono tutte in maiuscolo.

Strategy Pattern – Personaggi

Viene fatta la distinzione tra:

- **Personaggi giocabili** (*Player*): controllati dall'utente.
- **Nemici** (*Enemy*): personaggi non giocabili contro cui i giocatori si scontrano; possono esistere in più tipologie all'interno della stessa mappa.

Per gestire questa varietà è stato adottato il **Design Pattern Strategy**: poiché movimento e attacchi possono variare tra i personaggi giocabili, sono state create due interfacce/classi astratte rispettivamente per *attacco* e *movimento* di tutti i `Character`. Esempio:

```
public interface IFightStrategyEnemy{
    void initAttack(double deltatime, ACharacterEnemy enemy,
    ACharacterPlayable player);
    void normalAttack(double deltatime, ACharacterEnemy enemy,
     ACharacterPlayable player);
}
public class CommonAttackFireWeaponEnemy extends ACommonAttack
implements IFightStrategyEnemy {
    //...
    @Override
    public void normalAttack(...){//...}
    @Override
    public void initAttack(...) {//...}
}
```

Builder – Player ed Enemy

Viene adottato un Builder per la creazione del `Player` e dell'`Enemy` per separare come un oggetto viene costruito da cosa rappresenta. Esempio:

```
public abstract class ACharacterBuilder<T extends ACharacter> {

    protected EConcreteWeapon concrete_weapon;
    protected EProjectileType projectile_type;
    protected EConcreteMovement concrete_movement;
    protected EMovementType movement_type;
    protected EWeaponType weapon_type;
    protected InputManager input_manager;

    protected static final Map<EConcreteWeapon,
    Supplier<AWeapon>> weapon_registry = Map.of(
    EConcreteWeapon.NORMAL_PISTOL, () -> new FireWeaponFactory().createWeapon(
    EConcreteWeapon.NORMAL_PISTOL)
    );

    protected static final Map<EConcreteMovement, Function<InputManager,
    IMovementStrategy>> movement_registry = Map.of(
            EConcreteMovement.SIX_WAY, im -> new SixWaySmoothlyMovement(im),
            EConcreteMovement.ONE_WAY,  im -> new OneWayMovement()
    );

    public ACharacterBuilder<T> weaponType(EWeaponType w) { this.weapon_type = w;
    return this; }
    //per ogni variabile...

    protected void applyWeapon(ACharacter character) {//...}
    protected void applyMovement(ACharacter character){//...}
    protected abstract void applyFightStrategy(T character);
    public abstract T build();
```

```
}

public class EnemyBuilder extends ACharacterBuilder<ACharacterEnemy> {
    private static final Map<EProjectileType, Function<AFireWeapon, IFightStrategyEnemy>>
    projectile_registry = Map.of(
            EProjectileType.NORMAL, fw -> new CommonAttackFireWeaponEnemy(fw,
 new NormalProjectileFactory())
    );

    @Override
    protected void applyFightStrategy(ACharacterEnemy enemy) {
        if (projectile_type == null) return;
        enemy.setFight_strategy_enemy(projectile_registry.get(projectile_type)
        .apply((AFireWeapon) enemy.getWeapon()));
    }

    @Override
    public ACharacterEnemy build() {
        ACharacterEnemy enemy = new Enemy();
        applyWeapon(enemy);
        applyFightStrategy(enemy);
        applyMovement(enemy);
        return enemy;
    }
}
```

Factory Method – Armi e Personaggi

- Viene applicato il **Factory Method** per la creazione di armi e proiettili.
- Viene applicato il **Factory Method** per la creazione di nemici e personaggi giocabili.

Ogni personaggio può possedere più armi e quindi più attacchi: esiste un unico `Enemy` e un unico `Player`, che modificano il proprio movimento e attacco in base all'arma e alla strategia di movimento scelta. Le armi e il movimento sono progettati per essere il più generici possibile, in modo da poter essere riutilizzati sia per `Player` che per `Enemy`.

Le armi dispongono di un attacco standard, ma l'implementazione è differente per `Player` ed `Enemy`: vengono quindi create due classi distinte per implementare l'attacco.

```
public abstract class ACharacterEnemyFactory {
    public abstract ACharacterEnemy createEnemy(EWeaponType weapon_type,
    EConcreteWeapon concrete_weapon, EProjectileType projectile_type,
    EMovementType movement_type, EConcreteMovement concrete_movement);
}
public class EnemyFactory extends ACharacterEnemyFactory {
    @Override
    public ACharacterEnemy createEnemy(...) {
        return new EnemyBuilder()
                .weaponType(weapon_type)
                .concreteWeapon(concrete_weapon)
                .projectileType(projectile_type)
                .movementType(movement_type)
                .concreteMovement(concrete_movement)
```

```
                .build();
    }
}
```

Iterator – Proiettili

Viene adottato il design pattern Iterator per il tipo di proiettili, poichè un'arma può avere diversi tipi di proiettilie e rieco a scorrerli senza esporre la struttura interna di essi. Esempio:

```
public class ProjectileIterator implements Iterator<AProjectile> {
    private final List<AProjectile> projectiles;
    private int position = 0, last_returned_index = -1;

    public ProjectileIterator(List<AProjectile> projectilesList) {
        this.projectiles = projectilesList;
    }

    @Override
    public boolean hasNext() {
        return position < projectiles.size();
    }

    @Override
    public AProjectile next() {
        if (!hasNext()) {
            throw new NoSuchElementException("No more projectiles");
        }
        last_returned_index = position;
        return projectiles.get(position++);
    }

    @Override
    public void remove() {
        if (last_returned_index < 0) {
            throw new IllegalStateException("next() must be called before remove()");
        }
        projectiles.remove(last_returned_index);
        position = last_returned_index;
        last_returned_index = -1;
    }
}
```

MVC – Menù e UI

Viene adottato il **Design Pattern MVC** (Model–View–Controller) per separare logica, UI e interazione relativamente al menù e all'interfaccia utente. Esempio:

```
public class GameView {
    //...
    public GameView(double width, double height) {
        menu_layout = new VBox(20);
        menu_layout.setAlignment(Pos.CENTER);
```

```
        menu_layout.getStyleClass().add("vbox-menu");
        //...
    }
//...
}
public class GameModel {
    private IGameState game_state;
    public GameModel() {this.game_state = new MenuState();}
    public IGameState getGame_state() {return game_state;}
    public void startGame() {game_state.start(this);}
    //...
}
public class GameController {
    private GameModel model;
    private GameView view;
    private Stage stage;
    private CommandRegistry registry;

    public GameController(Stage stage) {//...}
    private void initController() {//...}
    public void startGame() {
        model.startGame();
        Group game_root = new Group();
        GameScene game_scene = new GameScene(...);
        stage.setScene(game_scene);
        GameUpdate gameUpdate = new GameUpdate(game_root);
        gameUpdate.startGameLoop(stage,game_scene);
        gameUpdate.setGame_controller(this);
        //...
    }//...
}
```

State Pattern – Game Loop e UI

Il **Design Pattern State** viene utilizzato per le fasi del game loop e per la gestione degli stati dell'UI. Esempio:

```
public interface IGameState {
    void start(GameModel context);
    void update(GameModel context);
    void stop(GameModel context);
}
public class StartState implements IGameState{
    @Override
    public void start(GameModel context) {
        System.out.println("Il gioco è iniziato.");
    }
    @Override
    public void update(GameModel context) {}
    @Override
    public void stop(GameModel context) {
    context.setGame_state(new QuitState(context));}
}
```

Singleton – HUD

Il **Singleton** viene applicato alla classe `HUD` e `EGameImages`. Esempio:

```
public class HUD {
    private volatile static HUD unique_instance;
    public static void  getInstance(){
        if(unique_instance == null){
            synchronized (HUD.class){
                if(unique_instance == null){
                    unique_instance = new HUD();
                }
            }
        }
    }
    //...
}
```

Command Pattern – Tasti UI

Il **Design Pattern Command** viene applicato per la gestione dei vari tasti dell'UI. Esempio:

```
public interface Command {
    void execute();
}
public class CommandRegistry {
    private final Map<String, Command> commands = new HashMap<>();
    public void register(String name, Command command) { commands.put(name, command); }
    public Command get(String name) { return commands.get(name); }
}
public class StartGameCommand implements Command {
    private final GameController controller;
    public StartGameCommand(GameController controller) {
        this.controller = controller;
    }
    @Override
    public void execute() {
        controller.startGame();
    }
}
```

1. Rispetto dei Principi SOLID

L'applicazione dei Design Pattern garantisce il rispetto quasi totale dei principi SOLID:

- **S - Single Responsibility Principle (SRP):** Ogni classe ha una responsabilità unica. L'uso di `MovementHandler`, `AttackHandler` e il pattern **MVC** assicura che la logica di business, la gestione degli input e la visualizzazione siano isolate.
- **O - Open/Closed Principle (OCP):** Grazie allo **Strategy Pattern** e al **Factory Method**, il sistema è aperto all'estensione (nuove armi o nemici) ma chiuso alla modifica del codice esistente.

- **L - Liskov Substitution Principle (LSP):** L'uso di interfacce (`I`) e classi astratte (`A`) permette di trattare `Player` ed `Enemy` in modo polimorfico senza compromettere la stabilità del sistema.
- **I - Interface Segregation Principle (ISP):** La separazione delle interfacce per movimento e attacco evita che le classi implementino metodi irrilevanti per il loro scopo specifico.
- **D - Dependency Inversion Principle (DIP):** Il Game Loop dipende da astrazioni (interfacce) e non da implementazioni concrete, facilitato dall'uso dei Factory.

2. Principi di Clean Code

Il progetto adotta diverse pratiche per favorire la leggibilità e la manutenibilità:

Nomenclatura Significativa: Uso di prefissi (*A, I, E*) e convenzioni rigorose (*camelCase* per i metodi, *underscore* per le variabili) che rendono il codice auto-esplicativo.

Principio di Least Knowledge (Legge di Demetra): In generale viene applicato il *Principle of Least Knowledge*, ispirato al Design Pattern **Facade**, per favorire il clean code e ridurre le dipendenze tra moduli minimizzando le interazioni dirette tra moduli distanti.

Encapsulation: L'uso del pattern **Iterator** per i proiettili protegge la struttura dati interna della collezione.

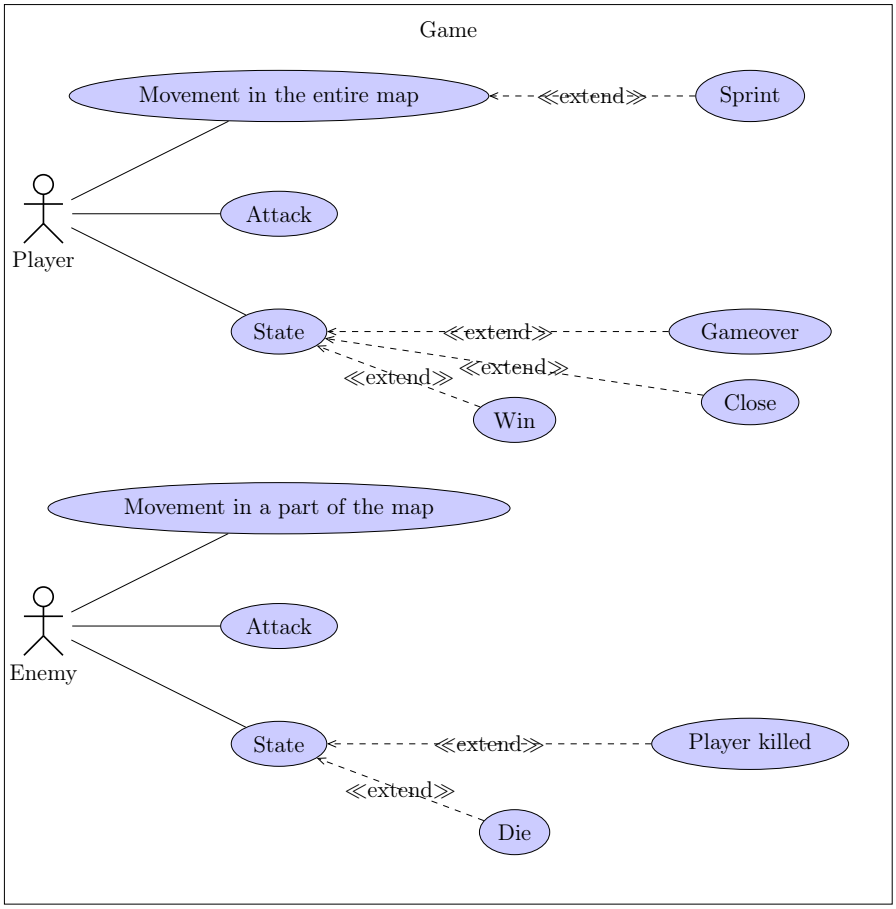
Separazione delle Creazione: L'uso dei pattern **Builder** e **Factory** separa la logica di costruzione dell'oggetto dalla sua logica di utilizzo.

3. Clean Architecture e Stratificazione

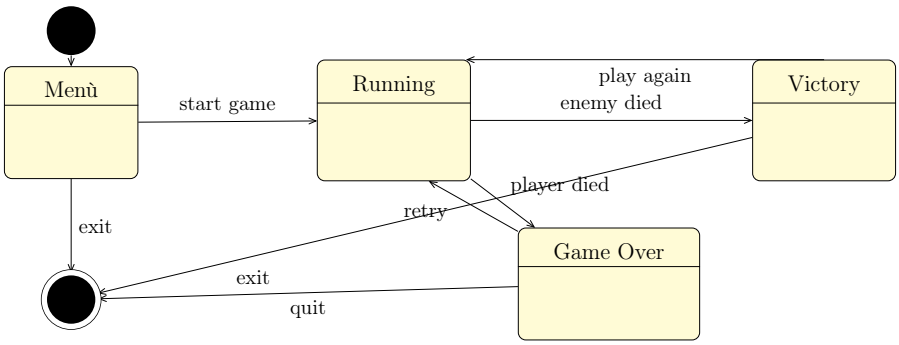
L'architettura del gioco riflette una chiara separazione a strati:

- **Indipendenza dalla UI:** Il pattern **MVC** e il **Command Pattern** isolano la logica decisionale dalla rappresentazione grafica.
- **Gestione dello Stato:** Il **State Pattern** nel Game Loop crea confini netti tra le fasi del gioco (Menu, In-Game, Game Over).
- **Disaccoppiamento dei Moduli:** Gli *Handler* fungono da orchestratori per specifici casi d'uso, simulando gli strati centrali della Clean Architecture.

UML – Use Case Diagram



UML – State Diagram



Descrizione

Il diagramma definisce il flusso di navigazione e il ciclo di vita del software:

- **Accesso:** Il sistema parte dal `Menù`, dove l'utente sceglie se avviare il gioco (`Running`) o uscire (`exit`).
- **Gameplay:** Nello stato `Running`, il gioco prosegue fino alla vittoria (`enemy died`) o alla sconfitta (`player died`).

- **Esiti:** Gli stati `Victory` e `Game Over` gestiscono la fine della sessione, permettendo di rigiocare immediatamente (`retry/play again`) o chiudere l'applicazione.

Una struttura che separa configurazione, esecuzione e gestione dei risultati, garantendo la continuità del loop di gioco.

UML – Activity Diagram

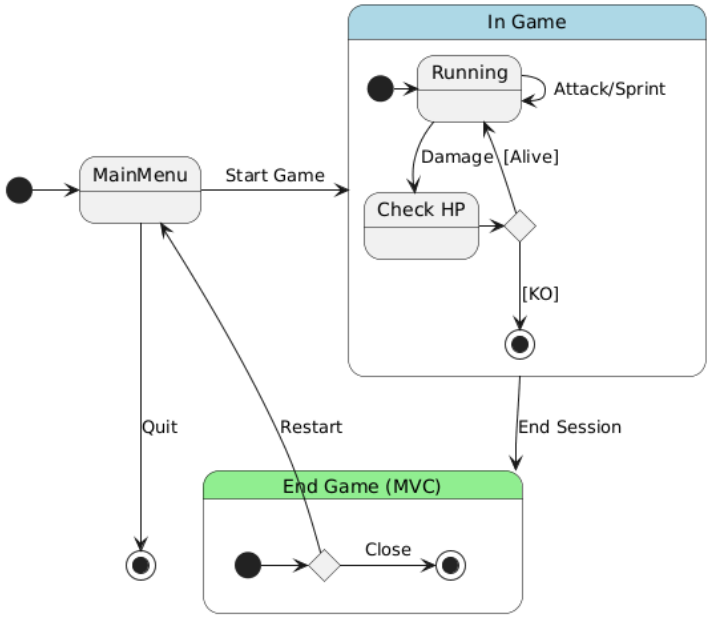


Figura 1: Activity Diagram

Descrizione

Il diagramma illustra gli stati principali dell'applicazione e le transizioni che regolano l'esperienza utente, dal lancio alla chiusura del software:

- **Gestione Menu:** Il sistema inizia nel `MainMenu`, fungendo da snodo principale per l'avvio del gameplay o l'uscita definitiva dall'applicazione.
- **Logica di Gameplay (In Game):** Rappresenta il nucleo interattivo. Lo stato `Running` gestisce le azioni continue (attacco/sprint). Qualsiasi danno subito innesca un passaggio allo stato di verifica (`CheckStatus`), dove un nodo decisionale (`choice`) determina se ripristinare il controllo al giocatore o terminare la sessione in caso di sconfitta o vittoria.
- **Ciclo Post-Partita (End Game):** Una volta conclusa la sessione di gioco, il sistema passa alla schermata di `End Game`. Qui l'utente può scegliere, tramite un punto di decisione, se reinizializzare il ciclo tornando al `MainMenu` o chiudere l'applicazione.

Il diagramma garantisce la corretta gestione del ciclo di vita del gioco, separando nettamente la fase di input attivo dalla gestione dei risultati e dei menu.

UML – Class Diagram

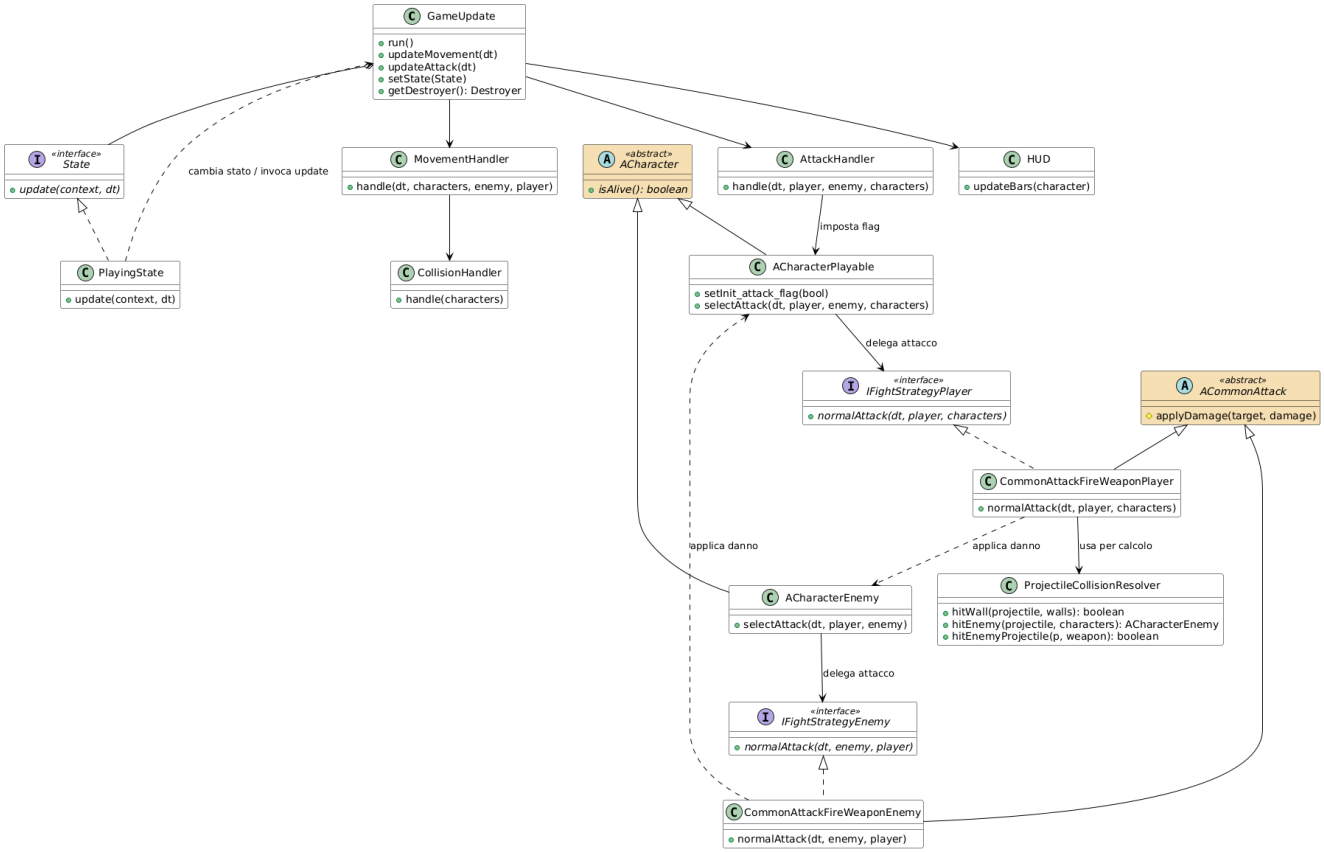


Figura 2: Class Diagram

Descrizione

L'architettura proposta segue un approccio modulare basato sulla separazione delle responsabilità e sull'estensibilità. I punti cardine sono:

- **State Pattern (Core Logic):** Il controllo è delegato a `PlayingState`. L'uso dell'interfaccia `State` permette al `GameUpdate` di cambiare comportamento (es. passare da gioco a menu) senza modificare la struttura del loop principale.
- **Disaccoppiamento tramite Handler:** La logica di aggiornamento è suddivisa in componenti specializzati (`MovementHandler`, `AttackHandler`). Il `MovementHandler` delega ulteriormente la fisica al `CollisionHandler`, isolando i calcoli di movimento dalle verifiche di collisione.
- **Strategy Pattern (Combattimento):** Le strategie d'attacco (`IFightStrategy`) sono separate dalle classi dei personaggi. Questo permette di variare le meccaniche di attacco (es. proiettili o mischia) a runtime semplicemente scambiando l'implementazione della strategia, garantendo elevata flessibilità.
- **Utility e UI:** La classe `ProjectileCollisionResolver` centralizza la logica geometrica dei proiettili, riducendo la ridondanza. L'interfaccia utente (`HUD`) è mantenuta passiva, aggiornata direttamente dal ciclo di gioco per garantire la coerenza dei dati visualizzati.

Il sistema applica rigorosamente i principi SOLID, facilitando l'aggiunta di nuovi contenuti (nemici, armi o stati) senza alterare il codice preesistente.

UML – Sequence Diagram

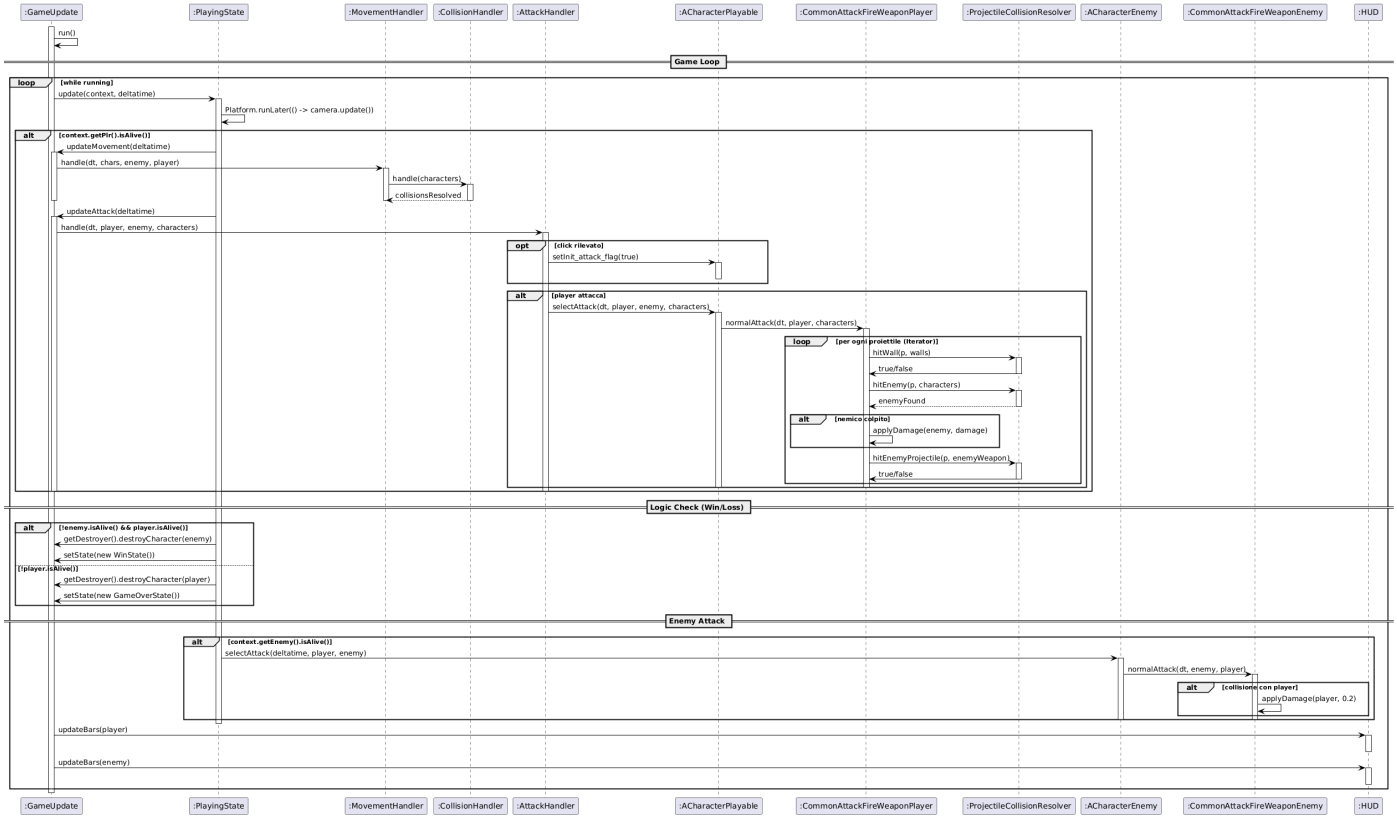


Figura 3: Sequence Diagram

Descrizione

Il diagramma illustra il flusso iterativo del `GameLoop`, orchestrato dalla classe `GameUpdate` che delega l'aggiornamento della logica allo stato corrente (`PlayingState`). Il processo si divide in quattro fasi principali:

- **Gestione Fisica e Input:** Se il giocatore è in vita, vengono invocati i gestori del movimento (`MovementHandler`) e delle collisioni. Successivamente, l'`AttackHandler` processa l'input (click del mouse) per inizializzare o continuare la sequenza di attacco.
- **Risoluzione dei Combattimenti:** La logica degli attacchi è delegata a classi strategia (`CAP` per il player, `CAE` per il nemico). Per il giocatore, un iteratore gestisce ogni proiettile attivo, verificando tramite il `PCR` (Collision Resolver) impatti con muri, nemici o proiettili avversari e applicando il relativo danno.
- **Verifica delle Condizioni di Terminazione:** Al termine di ogni ciclo, lo stato controlla la vitalità dei personaggi. Se uno dei due soccombe, viene invocato il `Destroyer` per la rimozione del nodo grafico e viene eseguito il cambio di stato verso `WinState` o `GameOverState`.
- **Aggiornamento Grafico:** L'ultima fase del ciclo prevede l'aggiornamento delle barre della salute nell'`HUD` per entrambi i personaggi, garantendo il feedback immediato all'utente.

Nota tecnica: L'uso di `Platform.runLater` per l'aggiornamento della camera indica una gestione asincrona del rendering rispetto alla logica di gioco.